

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ СИСТЕМ
УПРАВЛЕНИЯ И РАДИОЭЛЕКТРОНИКИ (ТУСУР)
Кафедра экономической математики, информатики и статистики (ЭМИС)

ДИНАМИЧЕСКИЕ СТРУКТУРЫ ДАННЫХ. ОДНОСВЯЗНЫЕ СПИСКИ

Отчёт по практической работе по дисциплине «Программирование»

Студент гр.594-2

А А. А. Танцеров

“ 23 ” Марта 2025 г.

Руководитель канд. физ.- мат. наук,
доцент каф. ЭМИС ТУСУР

_____ Е. А. Шельмина

“ ” _____ 2025 г.

Лабораторная работа №8

«Динамические структуры данных. Односвязные списки»

Цель работы: получить практические навыки разработки программ на языке C++ с использованием динамических структур данных.

Динамические структуры данных — это структуры, в которых объём памяти не является фиксированным. В таких структурах может храниться произвольное количество элементов. Размер динамической структуры ограничен только объёмом оперативной памяти компьютера.

Односвязный список (однонаправленный связный список) — это структура данных, состоящая из элементов одного типа, связанных между собой последовательно посредством указателей.

В отличие от статических структур (например, массивов), размер которых задаётся заранее, динамические структуры данных позволяют управлять памятью во время выполнения программы.

Главное преимущество — возможность гибкого использования памяти, без жёстко заданного размера.

Односвязный список – это набор элементов (узлов), где каждый узел хранит два элемента.

Первый элемент списка называется головой (head), последний — хвостом (tail). У последнего элемента указатель равен nullptr (нулевой указатель), так как за ним нет других узлов.

В C++ узел списка обычно представляют в виде структуры. Это означает, что каждый объект Node содержит число (data) и ссылку (next) на следующий элемент.

Задание 1. Организовать линейный односвязный список. Для решения задачи реализовать все необходимые функции (создание списка, добавление элемента в список, печать элементов списка, удаление элементов и т.д.). Написать функцию, которая удаляет из списка элементы, входящие в него только один раз.

Создаём структуру Node, включающую в себя элемент и указатель на следующий элемент. По аналогии с тарелками мы можем заполнять стек только сверху, не изменяя середины и низа. Для подобного потребуется переписывать весь стек. Выводим стек путём поочерёдного вывода временной переменной, хранящую в себе элемент стека, меняя указатель на следующий. Для удаления элемента из стека проверяем его местоположение, если он находится в начале, то во временную переменную передаём её значение, а указатель переносим на следующий элемент и удаляем временный. В противном случае находим местоположение элемента, создаём указатель на данный элемент, удаляем элемент, сдвигаем переменную на две «тарелки» вперёд. В функции main вызываем все пользовательские функции, выводим результат (рисунок 1).

The screenshot shows a C++ program in a code editor. The code defines a linked list structure with a 'Node' struct containing an integer 'elem' and a pointer 'next'. Functions include 'full_node' to create nodes, 'print' to traverse and print the list, and 'delete_elem' to remove a specific element. The 'main' function initializes the list with values 20, 30, 10, 20, 40, 20, 30, 20 and then deletes the element 10. The output window shows the list before and after deletion: '50 -> 10 -> 40 -> 20 -> 10 -> 30 -> 20 -> nullptr' and '10 -> 20 -> 10 -> 20 -> nullptr'.

```

1 #include <iostream>
2 #include <vector>
3 using namespace std;
4 struct Node {
5     int elem;
6     Node* next;
7 };
8 void full_node(Node*& head, int x) {
9     Node* a = new Node(x, head);
10    head = a;
11 }
12 void print(Node* head) {
13     Node* temp = head;
14     while (temp) {
15         cout << temp->elem << " -> ";
16         temp = temp->next;
17     }
18     cout << "nullptr" << endl;
19 }
20 void delete_elem(Node*& head) {
21     if (!head) return;
22     vector<int> values;
23     vector<int> counts;
24     Node* temp = head;
25     while (temp) {
26         auto x = find(values.begin(), values.end(), temp->elem);
27         if (x != values.end()) {
28             counts[x - values.begin()]++;
29         }
30         else {
31             values.push_back(temp->elem);
32             counts.push_back(1);
33         }
34         temp = temp->next;
35     }
36     while (head && counts[find(values.begin(), values.end(), head->elem) - values.begin()] == 1) {
37         Node* temp = head;
38         head = head->next;
39         delete temp;
40     }
41     if (!head) return;
42     temp = head;
43     while (temp->next) {
44         int index = find(values.begin(), values.end(), temp->next->elem) - values.begin();
45         if (counts[index] == 1) {
46             Node* toDelete = temp->next;
47             temp->next = temp->next->next;
48             delete toDelete;
49         }
50         else {
51             temp = temp->next;
52         }
53     }
54 }
55 int main() {
56     setlocale(LC_ALL, "RU");
57     Node* head = nullptr;
58     full_node(head, 20);
59     full_node(head, 30);
60     full_node(head, 10);
61     full_node(head, 20);
62     full_node(head, 40);
63     full_node(head, 10);
64     full_node(head, 20);
65     print(head);
66     delete_elem(head);
67     print(head);
68 }

```

Консоль отладки Microsoft VI

```

50 -> 10 -> 40 -> 20 -> 10 -> 30 -> 20 -> nullptr
10 -> 20 -> 10 -> 20 -> nullptr

```

C:\Users\atant\source\repos\emae8_1\x64\Debug\emae8_1.exe
Нажмите любую клавишу, чтобы закрыть это окно:

Рисунок 1 – Скриншот выполненного первого задания

Вывод: получены практические навыки разработки программ на языке C++ с использованием динамических структур данных.